

Lista 1 — Resolução Completa (Listas em Prolog)

Exercício 1 — analisa_lista/1

```
analisa_lista([]) :-  
    write('A lista esta vazia'), nl.  
analisa_lista([H|T]) :-  
    write('A cabeca da lista eh: '), write(H), nl,  
    write('A cauda da lista eh: '), write(T), nl.
```

Ideia: o padrão `[H|T]` já separa cabeça e cauda automaticamente na unificação. O caso da lista vazia precisa de uma cláusula própria porque `[]` não casa com `[H|T]`.

```
?- analisa_lista([dog,cat,horse,cow]).  
A cabeca da lista eh: dog  
A cauda da lista eh: [cat,horse,cow]  
true.
```

```
?- analisa_lista([]).  
A lista esta vazia  
true.
```

Exercício 2 — remove_duplicados/2

```
pertence(X, [X|_]).  
pertence(X, [_|T]) :- pertence(X, T).  
  
remove_duplicados([], []).  
remove_duplicados([H|T], R) :- pertence(H, T), !, remove_duplicados(T, R).  
remove_duplicados([H|T], [H|R]) :- remove_duplicados(T, R).
```

Ideia: percorre a lista da esquerda para a direita. Para cada `H`, verifica se ele **ainda aparece mais à frente** (`pertence(H,T)`). Se aparecer, descarta o `H` atual (mantém apenas a última ocorrência) — é por isso que o corte `!` existe: sem ele, o Prolog também tentaria a terceira cláusula em backtracking e duplicaria resultados.

```
?- remove_duplicados([a,b,a,c,d,d], List).  
List = [b, a, c, d].
```

Trace resumido: - `a` aparece de novo depois → descarta o primeiro `a` - `b` não aparece mais → mantém - segundo `a` não aparece mais → mantém - `c` não aparece mais → mantém - primeiro `d` aparece de novo → descarta - último `d` não aparece mais → mantém

Exercício 3 — troca/4

```
troca([], _, _, []).
troca([H|T], X, Y, [Y|L]) :- H == X, !, troca(T, X, Y, L).
troca([H|T], X, Y, [H|L]) :- troca(T, X, Y, L).
```

Ideia: cada elemento é comparado com X. Se for igual, substitui por Y; senão mantém. O corte impede que o Prolog também tente a terceira cláusula quando $H == X$ já teve sucesso (evitaria resultados duplicados/alternativos).

```
?- troca([1,2,3,4,3,5,6,3], 3, x, List).
List = [1, 2, x, 4, x, 5, 6, x].
```

Exercício 4 — potencia/2 (conjunto potência)

```
concat([], L, L).
concat([H|T], L, [H|R]) :- concat(T, L, R).

adiciona(_, [], []).
adiciona(X, [H|T], [[X|H]|R]) :- adiciona(X, T, R).

potencia([], [[]]).
potencia([H|T], P) :-
    potencia(T, PT),
    adiciona(H, PT, ComH),
    concat(ComH, PT, P).
```

Ideia central: $P(\text{conjunto vazio}) = [[]]$. Para um conjunto $[H|T]$, o conjunto potência é: todos os subconjuntos do resto (PT, sem H) **mais** todos esses mesmos subconjuntos com H adicionado (ComH).

```
?- potencia([a,b,c], P).
P = [[a,b,c], [a,b], [a,c], [a], [b,c], [b], [c], []].
```

Trace resumido (construção de baixo para cima): - potencia([], [[]])
- potencia([c], [[c], []]) - potencia([b,c], [[b,c], [b], [c], []]) -
potencia([a,b,c], [[a,b,c], [a,b], [a,c], [a], [b,c], [b], [c], []])

Exercício 5 — mais_longa/2

Versão do arquivo original:

```
mais_longa([], [_|_]).
mais_longa([_|T1], [_|T2]) :- mais_longa(T1, T2).
```

Funciona para os exemplos pedidos, mas tem uma limitação: exige que a **primeira lista** eventualmente chegue a exatamente um elemento — ou seja, ela falha silenciosamente se a primeira lista for vazia desde o início (`mais_longa([], [a])` deveria ser verdadeiro, mas essa versão falha).

Versão robusta (recomendada):

```
mais_longa([], [_|_]).
mais_longa([_|T1], [_|T2]) :- mais_longa(T1, T2).
```

Agora o caso base é “lista 1 vazia e lista 2 tem pelo menos 1 elemento a mais”, cobrindo também o caso trivial.

```
?- mais_longa([dog,cat,snake],[giraffe,elephant,lion,tiger]).
true.
```

```
?- mais_longa([1,2,3,4,5], []).
false.
```

Exercício 6 — distancia/3

```
distancia((X1,Y1), (X2,Y2), D) :-
    DX is X2 - X1,
    DY is Y2 - Y1,
    D is sqrt(DX*DX + DY*DY).

?- distancia((0,0),(3,4),X).
X = 5.0.

?- distancia((-2.5,1),(3.5,-4),X).
X = 7.810249675906654.
```

Exercício 7 — quadrado/2

```
quadrado(N, C) :- repete_linhas(N, N, C).

repete_linhas(0, _, _) :- !.
repete_linhas(I, N, C) :-
    I > 0,
    repete_colunas(N, C),
    nl,
    I1 is I - 1,
    repete_linhas(I1, N, C).

repete_colunas(0, _) :- !.
```

```

repete_colunas(J, C) :-
    J > 0,
    write(C), write(' '),
    J1 is J - 1,
    repete_colunas(J1, C).

```

```

?- quadrado(5, '*').

```

```

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
true.

```

Exercício 8 — elemento_n/3

```

elemento_n([H|_], 1, H) :- !.
elemento_n([_|T], N, X) :- N > 1, N1 is N - 1, elemento_n(T, N1, X).

```

```

?- elemento_n([tiger,dog,teddy_bear,horse,cow], 3, X).
X = teddy_bear.

```

```

?- elemento_n([a,b,c,d], 27, X).
false.

```

Exercício 9 — media/2

```

soma_lista([], 0).
soma_lista([H|T], S) :- soma_lista(T, S1), S is S1 + H.

tamanho_lista([], 0).
tamanho_lista([_|T], N) :- tamanho_lista(T, N1), N is N1 + 1.

```

```

media(L, X) :-
    soma_lista(L, S),
    tamanho_lista(L, N),
    N > 0,
    X is S / N.

```

```

?- media([1,2,3,4], X).
X = 2.5.

```

Exercício 10 — minimo/2

```
minimo([X], X) :- !.
minimo([H|T], M) :-
    minimo(T, M1),
    ( H < M1 -> M = H ; M = M1 ).

?- minimo([4,6,8,3,5,7], Result).
Result = 3.
```

Exercício 11 — intervalo/3

```
intervalo(Inf, Sup, []) :- Inf > Sup, !.
intervalo(Inf, Sup, [Inf|T]) :- Inf <= Sup, Inf1 is Inf + 1, intervalo(Inf1, Sup, T).

?- intervalo(3, 11, X).
X = [3,4,5,6,7,8,9,10,11].

?- intervalo(7, 4, X).
X = [].
```

(Poderia também usar o `between/3` embutido + `findall`, mas aqui a versão recursiva “na mão” é mais didática para prova.)

Exercício 12 — mdc/3 (Algoritmo de Euclides)

```
mdc(X, 0, X) :- !.
mdc(X, Y, D) :- Y > 0, R is X mod Y, mdc(Y, R, D).

?- mdc(57, 27, X).
X = 3.
```

Trace: `mdc(57,27,D) → R is 57 mod 27 = 3 → mdc(27,3,D) → R is 27 mod 3 = 0 → mdc(3,0,D) → corta e retorna D = 3.`

Exercício 13 — ocorrencias/3

```
ocorrencias(_, [], 0).
ocorrencias(X, [X|T], N) :- !, ocorrencias(X, T, N1), N is N1 + 1.
ocorrencias(X, [_|T], N) :- ocorrencias(X, T, N).

?- ocorrencias(dog, [dog,frog,cat,dog,dog,tiger], N).
N = 3.
```

O corte na segunda cláusula é essencial: sem ele, ao casar **X** com o primeiro elemento, o Prolog ainda tentaria (em backtracking) a terceira cláusula, contando o mesmo elemento de novo.

Exercício 14 — divisores/2

```
divisores(N, L) :- findall(D, (between(1, N, D), 0 is N mod D), L).
```

```
?- divisores(30, X).
```

```
X = [1,2,3,5,6,10,15,30].
```

Ideia: `findall` coleta, para cada `D` entre 1 e `N` gerado por `between/3`, todos os valores em que `N mod D = 0`.